COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10-07-2026	**DATE SUBMITTED:**	10-9-2026
NAME:	Condino, Niel Vincent B. Guarin, Jan Mikaela C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11 TITLE

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

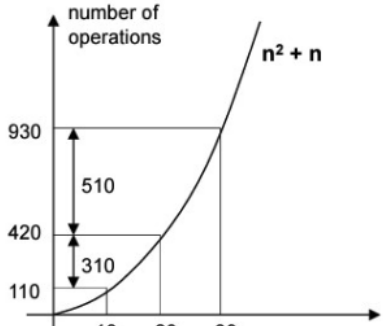
```
bool diff(int *x, int *y){
  for(int i = 0; i < y.length; i++){
     if(search(x, y[i]) == - 1){
     return false;
   }
  }
   return true;
  }
```

Performing worst-case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Calculation:
Operation 1 (Red highlight) - runs n times
Operation 2 (Yellow Highlight) - runs m times
Operation 3 (Blue Highlight) - runs n times

Given the above information, the total number of comparisons in the worst case is: **O((m*n)+n)**



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

Condino: As the input size grows, the runtime or number of operations of the program scales by the input array size squared plus the input array size. As shown in the graph, when the input size is 10, the expected number of operations becomes $10^2 + 10$, which is 110; when the input size is 20, it becomes 420.

Guarin: It shows that the number of operations increases as the size of the array gets bigger. When both arrays double in size, the number of comparisons becomes four times larger because the algorithm has quadratic growth. The function becomes slower for larger arrays.

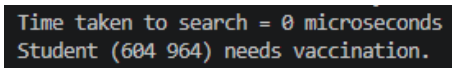
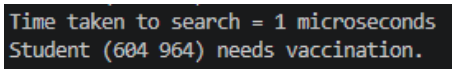
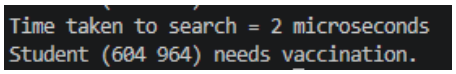
ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Table 11-1-A: Table for Runtime Analysis(Condino)

Input Size (N)	Execution Speed	Screenshot	Observation(s)
100	0 µs	Time taken to search = 0 microseconds Student (338 835) needs vaccination.	With an input of 100, the program runs in nanoseconds, so it logs 0 microseconds. The value can be around 500 nanoseconds - 999 nanoseconds
1000	1 µs	Time taken to search = 1 microseconds Student (338 835) needs vaccination.	With a 1000 input, the program took 1 microsecond to finish searching the vector using binary search,
100000	2 µs	Time taken to search = 2 microseconds Student (338 835) needs vaccination.	With an input of 100000, the program took 2 microseconds to search the vector using binary search.

Analysis: The function implements a binary search, which gives it a runtime complexity of O(log n). Analyzing further, each pass of the loop discards half of the array, giving log n time complexity. It does at most 3 comparisons: the (first < last) comparison. the (*mid == P) comparison, the (*mid > P) comparison. These comparisons all run at constant time. This means the algorithm runs at 3(log n) time. Comparing it with the experimental analysis, we can confirm that squaring the input at most doubles our runtime.

Table 11-1-B: Table for Runtime Analysis(Guarin)

Input Size (N)	Execution Speed	Screenshot	Observation(s)
100	0 µs		The program finished very quickly, recording 0 microseconds because the execution time was too short to register a full microsecond. This shows that processing a small number of elements requires very little time.
1000	1 µs		The program took 1 microsecond to finish execution. The slight increase in runtime shows that processing more elements can require additional time.
100000	2 µs		The program recorded 2 microseconds of execution time. Although the number of elements increased significantly, the measured runtime only increased slightly.

Analysis:

The program uses binary search, which halves the search range on every pass until it finds the target. Because of this, it only needs about log n steps, roughly 7 for 100, 10 for 1000, and 17 for 100000. My results of 0 µs, 1 µs, and 2 µs show almost no growth even when the input grew 1000x, which matches the theoretical O(log n). The 0 µs reading only means the search was too fast for the timer to show.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>

void randValArr(int arr[],int size){
    for(int i = 0; i < size; i++){
        arr[i] = rand()%100000;
    }
}

bool isUnique(int arr[],int size){
    for (int i = 0; i < size;i++){
        for (int j = i+1; j < size;j++){
            if (arr[i] == arr[j]){
                return false;
            }
        }
    }
    return true;
}

void runTest(int size){
    int* array = new int[size];
    randValArr(arr: array, size);

    auto begin: time_point = std::chrono::steady_clock::now();
    bool unique = isUnique(arr: array, size);
    auto end: time_point = std::chrono::steady_clock::now();

    std::cout << "Time taken to search array size " << size << " = "
              << std::chrono::duration_cast<
                  std::chrono::microseconds
                >(d: end - begin).count()
              << " microseconds\n";

    delete[] array;
}

int main(){
    srand(seed: time(timer: 0));
    const int sizes[4] = {[0]=100,[1]=1000,[2]=10000,[3]=1000000};
    for (int i = 0;i < 4;i++){
        runTest(size: sizes[i]);
    }
    return 0;
}
```

Figure 1-A: Finding unique element code (Condino)

The lines of code that will be analyzed is the function isUnique(). The outer loop iterates n times, scanning the whole array. The inner loop runs at most n-1 times. Inside of this inner loop is a comparison comparing arrays at index i and index j. With all of these steps in mind, the number of operations in this function is n*(n-1)*1 which simplifies to n(n-1) number of operations. Hence, this function has a relative runtime of O(n^2).

Table 2-A: Output Testing (Condino)

Input Size (N)	Screenshot
100	`Time taken to search array size 100 = 6 microseconds` `Array has unique elements`
1000	`Time taken to search array size 1000 = 584 microseconds` `Array has unique elements`
100000	`Time taken to search array size 100000 = 4694969 microseconds` `Array has unique elements`

The table above shows the experimental runtime of the function isUnique() on varying input sizes. At n = 100, the execution time is recorded to be about 6 microseconds. Multiplying n by 10 increased the recorded runtime to 584 microseconds; approximately raised to the 4th power of 6, or to be precise 6 raised to 3.5555109; a 97.33x increase in runtime. Increasing n to 100,000 puts the recorded runtime to 4694969 microseconds or about 4.69 seconds. This is around 584 raised to 2.41 or 6 raised to 8.5736. This runtime is around an 8000x increase from n = 1000 and 782,00x times increase from n =100. Compared to our theoretical analysis and experiment, the runtime increase from n=100 to n=1000 is approximately what we are expecting around a 100x times increase. This increase is calculated using n^2 -> (10n)^2->100n^2.

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>

void uniqueValArr(int arr[], int size){
    for (int i = 0; i < size; i++) arr[i] = i;
    for (int i = size - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

bool isUnique(int arr[], int size){
    for (int i = 0; i < size; i++){
        for (int j = i + 1; j < size; j++){
            if (arr[i] == arr[j]) return false;
        }
    }
    return true;
}

void runTest(int size){
    int* array = new int[size];
    uniqueValArr(array, size);

    auto begin = std::chrono::steady_clock::now();
    bool unique = isUnique(array, size);
    auto end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search array size " << size << " = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds\n";
    std::cout << (unique ? "Array has unique elements\n" : "Array has duplicates\n");

    delete[] array;
}

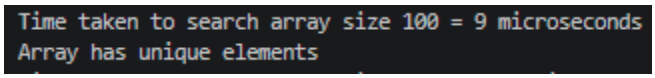
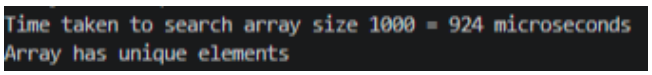
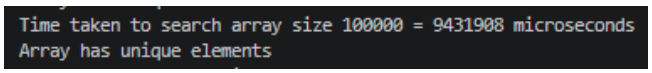
int main(){
    srand(time(0));
    const int sizes[3] = {100, 1000, 100000};
    for (int i = 0; i < 3; i++){
        runTest(sizes[i]);
    }
    return 0;
}
```

Figure 1-B: Finding unique element code (Guarin)

Analysis:

isUnique() function checks whether the elements in an array are unique by comparing each element with the other elements. The outer loop selects an element, while the inner loop compares it with the remaining elements to check if there are any duplicates. Since the code may compare many pairs of elements as the array grows, the number of operations increases significantly for larger inputs. Therefore, the function has time complexity of O(n^2), meaning that increasing the input size can make the program take much longer to finish.

Table 2-B: Output Testing (Guarin)

Input Size (N)	Screenshot
100	
1000	
100000	

Analysis:

The output shows the changes when the input size increases from 100 to 100,000. As the number of elements grows, the function needs to problem more comparisons to determine whether the array contains duplicate values. This is consistent with its theoretical time complexity of O(n^2), since the number of comparisons can increase approximately with the square of input size.

Problem 2:

```cpp
#include <iostream>
#include <chrono>

int rpower(int x,int n){
    if (n == 0) return 1;
    else return x*rpower(x, n: n-1);
}

int brpower(int x,int n){
    if (n==0) return 1;
    if (n%2){
        int y = brpower(x, n: (n-1)/2);
        return x*y*y;
    }
    else{
        int y = brpower(x, n: n/2);
        return y*y;
    }
}

void runTest(std::string funcName,int x,int n,int (*func)(int,int)){

    auto begin: time_point = std::chrono::steady_clock::now();
    func(x,n);
    auto end: time_point = std::chrono::steady_clock::now();

    std::cout << "Time taken to execute " << funcName << " = "
              << std::chrono::duration_cast<
                    std::chrono::nanoseconds
                 >(d: end - begin).count()
              << " nanoseconds\n";
}

int main(){
    const int sizes[3] = {[0]=100,[1]=1000,[2]=10000};
    for (int i = 0;i < 3;i++){
        runTest(funcName: "rpower",x: sizes[i],n: sizes[i],func: rpower);
    }

    for (int i = 0;i < 3;i++){
        runTest(funcName: "brpower",x: sizes[i],n: sizes[i],func: brpower);
    }
    return 0;
}
```

Figure 2-A: rpower and brpower function (Condino)

The 2 functions at interest here are rpower() and brpower(). The rpower() calculates power by recursively calling itself until it reaches n=0. Analyzing this function, The return line runs 1 time while the recursive call runs at n times. This gives the function a n+1 execution step or a runtime complexity of O(n).

The brpower() tried to half the input size in each iteration. The first return comparison and modulo comparison operation runs at 1 time. Each branch of the if statement halves the input size by half but calls the function recursively, then lastly a return statement. This gives approximately 2(log n) execution steps or an O(log n) time complexity.

Table 3-A: Outputs testing (Condino)

Input Size (N)	Screenshot
100	`Time taken to execute rpower = 300 nanoseconds` `Time taken to execute brpower = 90 nanoseconds`
1000	`Time taken to execute rpower = 9500 nanoseconds` `Time taken to execute brpower = 120 nanoseconds`
100000	`Time taken to execute rpower = 1505188 nanoseconds` `Time taken to execute brpower = 160 nanoseconds`

The table above shows the time taken for each input size.

The theoretical analysis shows that the rpower() should have a runtime complexity of O(n). However, plotting these values in a graph does not show a linear growth. 300 should grow to 3000 ideally when the input grew to 1000; 300,000 when it grew to 100,000 input size. The growth of runtime using experimental analysis suggests a different scenario is happening. Increasing the input to by a factor of 10 increases the runtime by 31x; Increasing the input by a factor of 1000 increases the runtime by around 5000x. It seems to me that it happens because of the background applications I have been running at the moment I run this code.

The theoretical analysis for brpower() suggests that the code should have a time complexity of O(log n). Running the experiment to see its runtime, As the input size grows by different factors of 10, the runtime slowly grows. Increasing the input size from 100 to 1000 increased the runtime by 1.33x; from 1000 to 100,000 also increased the runtime 1.33x. Comparing the runtime of brpower() and rpower() shows how big the gap is between linear time complexity and logarithmic time complexity.

```cpp
#include <iostream>
#include <chrono>

int rpower(int x, int n){
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}

int brpower(int x, int n){
    if (n == 0) return 1;
    if (n % 2){
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }
    else{
        int y = brpower(x, n / 2);
        return y * y;
    }
}

void runTest(const char* funcName, int x, int n, int (*func)(int, int)){
    auto begin = std::chrono::steady_clock::now();
    volatile int result = func(x, n);
    (void)result;
    auto end = std::chrono::steady_clock::now();

    std::cout << "Time taken to execute " << funcName << " = "
              << std::chrono::duration_cast<std::chrono::nanoseconds>(end - begin).count()
              << " nanoseconds\n";
}

int main(){
    const int sizes[3] = {100, 1000, 100000};
    for (int i = 0; i < 3; i++){
        std::cout << "N = " << sizes[i] << "\n";
        runTest("rpower", 1, sizes[i], rpower);
        runTest("brpower", 1, sizes[i], brpower);
    }
    return 0;
}
```

Figure 2-A: rpower and brpower function (Guarin)

Analysis: The rpower() function calculates a power by repeatedly calling itself with a smaller exponent until it reaches the base case, where the exponent is zero. Because the exponent decreases one step at a time, the function has a time complexity of O(n). In comparison, brpower() reduces the exponent by half during each recursive call, allowing it to reach the base case in fewer steps. This gives brpower() a time complexity of O(log n), making it more efficient for larger exponents because it performs fewer recursive calls.

Input Size (N)	Screenshot
100	Time taken to execute rpower = 640 nanoseconds Time taken to execute brpower = 189 nanoseconds
1000	Time taken to execute rpower = 21849 nanoseconds Time taken to execute brpower = 149 nanoseconds
100000	Time taken to execute rpower = 4152036 nanoseconds Time taken to execute brpower = 270 nanoseconds

Table 3-A: Outputs testing (Guarin)

Analysis: The output testing compares the execution times of rpower() and brpower() for input sizes of 100, 1,000, and 100,000. The rpower() function is expected to take more time as the input increases because it processes the exponent one step at a time. Meanwhile, brpower() reduces the exponent by half in each recursive call, so its execution time should increase more slowly. The experimental results can be compared with the theoretical analysis to observe the difference between linear and logarithmic time complexity.

Conclusion:

(Condino)
In this activity, I was able to enhance my skill on theoretical analysis of code runtime and space complexity. Each operation inside code takes time and these operations can vary hardware to hardware. Even without changing the hardware, the same code can have varying runtime based on when they are run. This is why theoretical analysis of code runtime and space complexity is important. This analysis gives us a relative runtime and space complexity in relation to the input of a program. This provides a much reliable way of how the function will scale along with its input. I also explored and compared the theoretical analysis and the experimental way by comparing my computed runtime complexity versus the actual runtime of the program. While it does not really scale with the input according to the computed analysis perfected but it was near that projected value so it still provides insight on how it still grows according to input. Overall, I aim to use this knowledge about analysis to further help with optimizing my code in the future to run on different kinds of hardware.

(Guarin)

Through this activity, I learned how to measure and analyze the runtime of algorithms using different input sizes. I understood how the code works through loops, comparisons, and recursive calls, and how these affect the execution time of a program. I also learned the difference between linear, quadratic, and logarithmic time complexity by analyzing functions such as isUnique(), rpower(), brpower(). Comparing theoretical and experimental results helped me to understand more why some algorithms understand better than others when the input size increases. However, there are still things that I need to improve such as calculating the number of operations, identifying time complexity, and explaining the difference between theoretical and actual execution times.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly(Trial Version)**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Fix grammar better than google docs native implementation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.